

请汇总pytorch里的常见函数和函数导数



在 PyTorch 中，常见的函数和它们的导数（如果适用）非常重要，特别是在深度学习中用于自动微分。以下是一些常见的函数以及它们的导数：

1. 加法（Addition）

- 函数: $f(x) = x + y$
- 导数: $f'(x) = 1$

python

复制代码

```
import torch
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = torch.tensor([4.0, 5.0], requires_grad=True)
z = x + y
z.backward()
print(x.grad) # tensor([1., 1.])
print(y.grad) # tensor([1., 1.])
```

2. 乘法（Multiplication）

- 函数: $f(x) = x \cdot y$
- 导数: $f'(x) = y$

python

复制代码

```
z = x * y
z.backward()
print(x.grad) # tensor([4., 5.])
print(y.grad) # tensor([2., 3.])
```

3. 幂运算（Power）

- 函数: $f(x) = x^n$
- 导数: $f'(x) = n \cdot x^{n-1}$

python

复制代码

```
z = x ** 2 # f(x) = x^2
z.backward()
print(x.grad) # tensor([4., 6.])
```

4. 线性变换（Linear Transformation）

- 函数: $f(x) = Ax + b$
- 导数: $f'(x) = A$

python

复制代码

```
A = torch.tensor([[2.0, 1.0], [1.0, 2.0]])
b = torch.tensor([1.0, 1.0])
z = torch.matmul(A, x) + b
z.backward(torch.ones_like(x)) # Set the gradient of output to 1
print(x.grad) # tensor([3., 5.])
```

5. ReLU（Rectified Linear Unit）

- 函数: $f(x) = \max(0, x)$
- 导数: $f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$

python

复制代码

```
x = torch.tensor([-1.0, 2.0], requires_grad=True)
z = torch.relu(x)
z.backward(torch.ones_like(x))
print(x.grad) # tensor([0., 1.])
```

6. Sigmoid 函数

- 函数: $f(x) = \frac{1}{1+e^{-x}}$
- 导数: $f'(x) = f(x) \cdot (1 - f(x))$

python

复制代码

```
x = torch.tensor([0.5, -1.5], requires_grad=True)
z = torch.sigmoid(x)
z.backward(torch.ones_like(x))
print(x.grad)  # tensor([0.2350, 0.1966])
```

7. Tanh 函数

- 函数: $f(x) = \tanh(x)$
- 导数: $f'(x) = 1 - f(x)^2$

python

复制代码

```
x = torch.tensor([1.0, -2.0], requires_grad=True)
z = torch.tanh(x)
z.backward(torch.ones_like(x))
print(x.grad)  # tensor([0.4199, 0.0707])
```

8. Softmax 函数

- 函数: $f(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$
- 导数: 对于 x_i , 导数为:

$$\frac{\partial f(x)_i}{\partial x_j} = f(x)_i \cdot (\delta_{ij} - f(x)_j)$$

其中 δ_{ij} 是 Kronecker delta。

python

复制代码

```
x = torch.tensor([2.0, 1.0, 0.5], requires_grad=True)
z = torch.softmax(x, dim=0)
z.backward(torch.ones_like(x))
print(x.grad)
```

9. LogSoftmax 函数

- 函数: $f(x)_i = \log\left(\frac{e^{x_i}}{\sum_j e^{x_j}}\right)$
- 导数: 类似于 Softmax, 只是对数形式。

python

复制代码

```
x = torch.tensor([2.0, 1.0, 0.5], requires_grad=True)
z = torch.log_softmax(x, dim=0)
z.backward(torch.ones_like(x))
print(x.grad)
```

10. Log 函数

- 函数: $f(x) = \log(x)$
- 导数: $f'(x) = \frac{1}{x}$

python

复制代码

```
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
z = torch.log(x)
z.backward(torch.ones_like(x))
print(x.grad)  # tensor([1., 0.5000, 0.3333])
```

11. 平方根 (Square Root)

- 函数: $f(x) = \sqrt{x}$
- 导数: $f'(x) = \frac{1}{2\sqrt{x}}$

python

复制代码

```
x = torch.tensor([4.0, 9.0], requires_grad=True)
z = torch.sqrt(x)
z.backward(torch.ones_like(x))
print(x.grad)  # tensor([0.2500, 0.1667])
```

12. 指数 (Exponentiation)

- 函数: $f(x) = e^x$
- 导数: $f'(x) = e^x$

python

复制代码

```
x = torch.tensor([1.0, 2.0], requires_grad=True)
z = torch.exp(x)
z.backward(torch.ones_like(x))
print(x.grad)  # tensor([2.7183, 7.3891])
```

13. Cos 和 Sin

- 函数:
 - $f(x) = \cos(x)$
 - $f'(x) = -\sin(x)$
 - $f(x) = \sin(x)$
 - $f'(x) = \cos(x)$

python

复制代码

```
x = torch.tensor([1.0, 0.5], requires_grad=True)
z = torch.cos(x)
z.backward(torch.ones_like(x))
print(x.grad)  # tensor([-0.5403, -0.3776])

z = torch.sin(x)
z.backward(torch.ones_like(x))
print(x.grad)  # tensor([ 0.8415,  0.4794])
```

14. 矩阵乘法（Matrix Multiplication）

- 函数: $f(x) = A \cdot x$
- 导数: 对于 x ，导数为矩阵 A 。

python

复制代码

```
A = torch.tensor([[1.0, 2.0], [3.0, 4.0]], requires_grad=True)
x = torch.tensor([1.0, 1.0], requires_grad=True)
z = torch.matmul(A, x)
z.backward(torch.ones_like(x))
print(x.grad)  # tensor([4., 6.])
```

总结

PyTorch 提供了广泛的数学函数，并且支持自动微分。上述例子展示了常见的数学函数及其导数，PyTorch 中的 `backward()` 方法可以自动计算并返回梯度。如果你使用 `requires_grad=True` 来创建张量，PyTorch 会记录操作并自动计算梯度，非常适合深度学习模型的反向传播过程。

通过利用 PyTorch 这些函数的自动微分功能，你可以方便地进行梯度计算，而不需要手动推导或实现这些导数。

这些导数函数是定理还是公理

这些导数函数本质上是微积分中的定理（或公式），而不是公理。

定理和公理的区别

- 公理（Axiom）：是某个数学理论的基本假设或起点，通常是显而易见、无法进一步证明的真理。公理作为出发点，用来推导其他结果。
- 定理（Theorem）：是通过逻辑推理或已有的公理和假设证明出来的数学命题。定理通常基于一定的公理、定义和之前的定理，经过严密的推导得出。

微积分中的导数公式

这些导数（例如加法法则、链式法则、幂函数的导数、对数函数的导数等）都是基于微积分的基本定理，具体来说，它们可以通过极限定义的微分操作得到。常见的导数公式不是公理，而是由数学家基于以下几种方式证明的：

- 定义导数：导数的定义是基于极限的思想，公式如：

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

这个定义可以用来推导不同函数的导数。

- 基本定理：通过微积分的基本定理，例如“导数的线性性”和“乘法规则”，可以推导出许多函数的导数公式。
- 导数法则：
 - 加法法则： $(f + g)'(x) = f'(x) + g'(x)$
 - 乘法法则（莱布尼茨法则）： $(f \cdot g)'(x) = f'(x) \cdot g(x) + f(x) \cdot g'(x)$
 - 链式法则： $\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$ 这些法则可以通过导数的定义和极限的性质推导出来。

例如：幂函数的导数公式

对于幂函数 $f(x) = x^n$ （其中 n 是常数），它的导数公式是通过极限定义导出的：

$$f'(x) = \lim_{h \rightarrow 0} \frac{(x + h)^n - x^n}{h}$$

这个导数公式是通过展开二项式，并应用极限求得的。这个公式本身是定理，因为它是通过推导得出的。

常见导数公式来源

- 基本函数的导数：**例如幂函数、指数函数、对数函数、三角函数等的导数，可以通过极限定义直接推导出来。
- 导数法则：**加法法则、乘法法则、链式法则等是微积分的基础法则，可以通过定义导数和极限操作来证明。

结论

因此，这些导数公式和法则是基于微积分的定义和定理，而不是公理。它们是通过推导和证明得到的数学结果，并且是微积分理论的一部分。

用numpy再次实现上面的函数导数



好的，下面是使用 `NumPy` 来实现一些常见函数及其导数的代码示例。我们将使用 **数值微分** 的方式来估算这些函数的导数，而不是符号求导。具体来说，我们使用 **有限差分法** 来近似导数，公式如下：

$$f'(x) \approx \frac{f(x + h) - f(x)}{h}$$

其中，`h` 是一个小的增量。下面是具体实现：

1. 加法函数（Addition）

- $f(x) = x + y$
- 导数: $f'(x) = 1$

python

```
import numpy as np

def add_func(x, y):
    return x + y

def add_func_derivative(x, y, h=1e-5):
    return (add_func(x + h, y) - add_func(x, y)) / h

x = np.array([2.0, 3.0])
y = np.array([4.0, 5.0])
grad = add_func_derivative(x, y)
print(grad) # 输出导数, 应该是 [1. 1.]
```

复制代码

2. 乘法函数（Multiplication）

- $f(x) = x \cdot y$
- 导数: $f'(x) = y$

python

```
def mul_func(x, y):
    return x * y

def mul_func_derivative(x, y, h=1e-5):
    return (mul_func(x + h, y) - mul_func(x, y)) / h

grad = mul_func_derivative(x, y)
print(grad) # 输出导数, 应该是 [4. 5.]
```

复制代码

3. 幂函数（Power）

- $f(x) = x^n$
- 导数: $f'(x) = n \cdot x^{n-1}$

python

```
def power_func(x, n):
    return x ** n

def power_func_derivative(x, n, h=1e-5):
    return (power_func(x + h, n) - power_func(x, n)) / h

grad = power_func_derivative(x, 2)
print(grad) # 输出导数, 应该接近 [4. 6.]
```

复制代码

4. ReLU 函数（Rectified Linear Unit）

- $f(x) = \max(0, x)$
- 导数:

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

python

复制代码

```
def relu_func(x):  
    return np.maximum(0, x)  
  
def relu_func_derivative(x, h=1e-5):  
    return (relu_func(x + h) - relu_func(x)) / h  
  
x = np.array([-1.0, 2.0])  
grad = relu_func_derivative(x)  
print(grad) # 输出导数, 应该是 [0. 1.]
```

5. Sigmoid 函数

- $f(x) = \frac{1}{1+e^{-x}}$
- 导数: $f'(x) = f(x) \cdot (1 - f(x))$

python

复制代码

```
def sigmoid_func(x):  
    return 1 / (1 + np.exp(-x))  
  
def sigmoid_func_derivative(x, h=1e-5):  
    return (sigmoid_func(x + h) - sigmoid_func(x)) / h  
  
x = np.array([0.5, -1.5])  
grad = sigmoid_func_derivative(x)  
print(grad) # 输出导数, 应该接近 [0.2350, 0.1966]
```

6. Tanh 函数

- $f(x) = \tanh(x)$
- 导数: $f'(x) = 1 - f(x)^2$

python

复制代码

```
def tanh_func(x):  
    return np.tanh(x)  
  
def tanh_func_derivative(x, h=1e-5):  
    return (tanh_func(x + h) - tanh_func(x)) / h  
  
x = np.array([1.0, -2.0])  
grad = tanh_func_derivative(x)  
print(grad) # 输出导数, 应该接近 [0.4199, 0.0707]
```

7. Log 函数

- $f(x) = \log(x)$
- 导数: $f'(x) = \frac{1}{x}$

python

复制代码

```
def log_func(x):  
    return np.log(x)  
  
def log_func_derivative(x, h=1e-5):  
    return (log_func(x + h) - log_func(x)) / h  
  
x = np.array([1.0, 2.0, 3.0])  
grad = log_func_derivative(x)  
print(grad) # 输出导数, 应该接近 [1., 0.5000, 0.3333]
```

8. 平方根（Square Root）

- $f(x) = \sqrt{x}$
- 导数: $f'(x) = \frac{1}{2\sqrt{x}}$

python

复制代码

```
def sqrt_func(x):  
    return np.sqrt(x)  
  
def sqrt_func_derivative(x, h=1e-5):  
    return (sqrt_func(x + h) - sqrt_func(x)) / h  
  
x = np.array([4.0, 9.0])  
grad = sqrt_func_derivative(x)  
print(grad)  # 输出导数, 应该接近 [0.2500, 0.1667]
```

9. 指数 (Exponentiation)

- $f(x) = e^x$
- 导数: $f'(x) = e^x$

python

复制代码

```
def exp_func(x):  
    return np.exp(x)  
  
def exp_func_derivative(x, h=1e-5):  
    return (exp_func(x + h) - exp_func(x)) / h  
  
x = np.array([1.0, 2.0])  
grad = exp_func_derivative(x)  
print(grad)  # 输出导数, 应该接近 [2.7183, 7.3891]
```

总结

在上面的代码中，我们通过 **有限差分法** 来计算每个函数的导数。每次计算时，我们都会选择一个小的步长 `h`，然后利用函数的差分近似导数。通过这种方法，尽管我们无法获得精确的符号导数，但我们可以获得足够接近真实值的数值解。

这些数值微分方法在深度学习和机器学习中广泛应用，尤其是在没有显式表达式求导的情况下，例如当使用自动微分引擎（如 PyTorch 和 TensorFlow）时。

